

Chapter 9-2

Solution Search Algorithms

Branch-and-Bound Algorithm

- 분기 한정 (Branch-and-Bound) 기법
 - Backtracking 기법은 깊이 우선 탐색 수행
 - 최적화 문제에 대해서는 최적해가 상태 공간 tree의 어디에 있는지 알 수 없으므로, tree에서 대부분의 노드를 탐색하여야 함
 - 입력의 크기가 커지면 해를 찾는 것은 거의 불가능
 - Branch-and-bound 기법은 이러한 단점을 보완하는 탐색 기법

Branch-and-Bound Algorithm

- 분기 한정 (Branch-and-Bound) 기법
 - Branch-and-bound 기법은 상태 공간 tree의 각 노드(상태)에 특정한 값(한정값)을 부여
 - 노드의 한정값을 활용하여 가지치기를 함으로써 backtracking 기법보다 빠르게 해를 찾음
 - Branch-and-bound 기법에서는 가장 우수한 한정값을 가진 노드를 먼저 탐색하는 최선 우선 탐색 (Best First Search)으로 해를 찾음

Branch-and-Bound Algorithm

- 분기 한정 (Branch-and-Bound) 기법의 효율적인 탐색 원리
 - 최적해를 찾은 후에 나머지 노드의 한정값이 최적해의 값과 같거나 나쁘면 더 이상 탐색하지 않음
 - 상태 공간 tree의 대부분의 노드가 문제의 조건에 맞지 않아서 해가 되지 못함
 - 최적해가 있을 만한 영역을 먼저 탐색함

Branch-and-Bound Algorithm

- 알고리즘

Branch-and-Bound(S) // S는 문제의 초기 상태

1. 상태 S의 한정값을 계산
2. activeNodes = {S} // 탐색되어야 하는 상태의 집합
3. bestValue = ∞ // 현재까지 탐색된 해 중의 최솟값
4. while (activeNodes $\neq \Phi$) {
5. $S_{\min}$ = activeNodes의 상태 중에서 한정값이 가장 작은 상태
6. $S_{\min}$ 을 activeNodes에서 제거
7. $S_{\min}$ 의 자식(확장 가능한) 노드 $S_1', S_2', \dots, S_k'$ 를 생성하고 각각의 한정값을
 계산함

Branch-and-Bound Algorithm

- 알고리즘

8. for $i=1$ to k // 확장한 각 자식 S_i' 에 대해서
9. if S_i' 의 한정값 $\geq$ bestValue
10. S_i' 를 가지치기 함 // 더 우수한 해가 없으므로
11. else if S_i' 가 완전한 해이고 S_i' 의 값 $<$ bestValue
12. bestValue = S_i' 의 값
13. bestSolution = S_i'
14. else
15. S_i' 를 activeNodes에 추가

Branch-and-Bound Algorithm

- TSP 적용

- Branch-and-bound 기법으로 문제의 최적해를 찾으려면, 먼저 각 상태에서 의 한정값을 계산하여야 함
- 한정값 계산을 위한 여행자 문제의 조건
 - 문제의 해는 주어진 시작점에서 출발하여 모든 다른 점을 1번씩만 방문하고 시작점으로 돌아와야 함
- 경로 상의 1개의 점 x 를 살펴보면, 다른 점에서 점 x 로 들어온 후에 점 x 를 떠나 또 다른 점으로 나감. 이를 점 x 의 한정값 계산에 활용

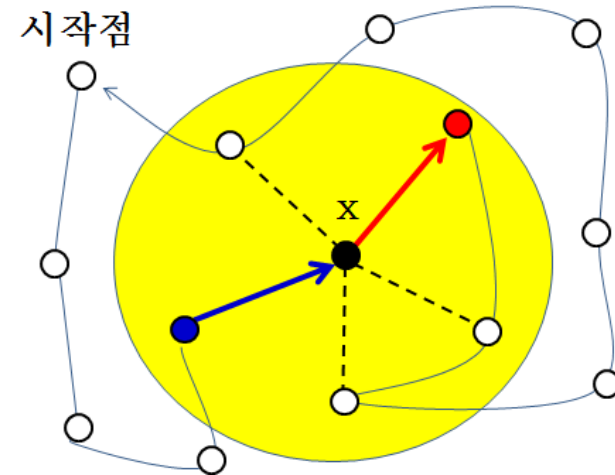
Branch-and-Bound Algorithm

- TSP의 한정값 계산 방법

- TSP에서 임의의 점 x 에서의 한정값 = 시작점부터 점 x 까지의 경로 길이 + 점 x 를 떠나서 남은 다른 점들을 1번씩만 방문하고 시작점으로 돌아오는 경로의 '예측' 길이

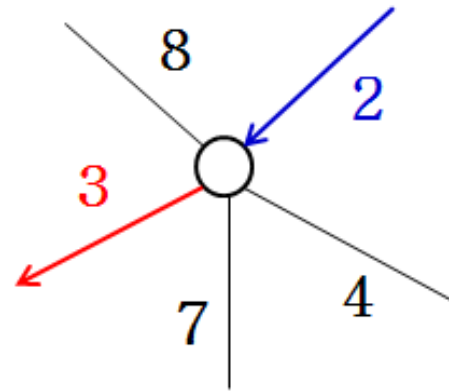
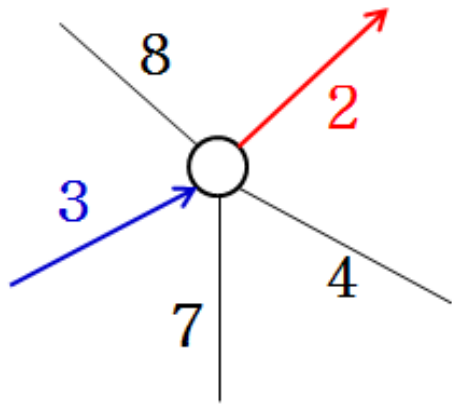
- TSP 문제는 최단 경로를 찾는 문제이므로 앞으로 방문해야 할 각 점 x 에 연결된 간선 중에서 가장 짧은 두 간선의 가중치의 평균의 합을 예측 길이를 계산하는 데에 사용

- 가중치의 합을 $\frac{1}{2}$ 로 곱하는 (평균을 내는) 이유는 한 점에서 나가는 간선은 인접한(다른) 점에서 들어오는 간선과 동일하므로
- 단, 소수점 이하의 숫자는 올림



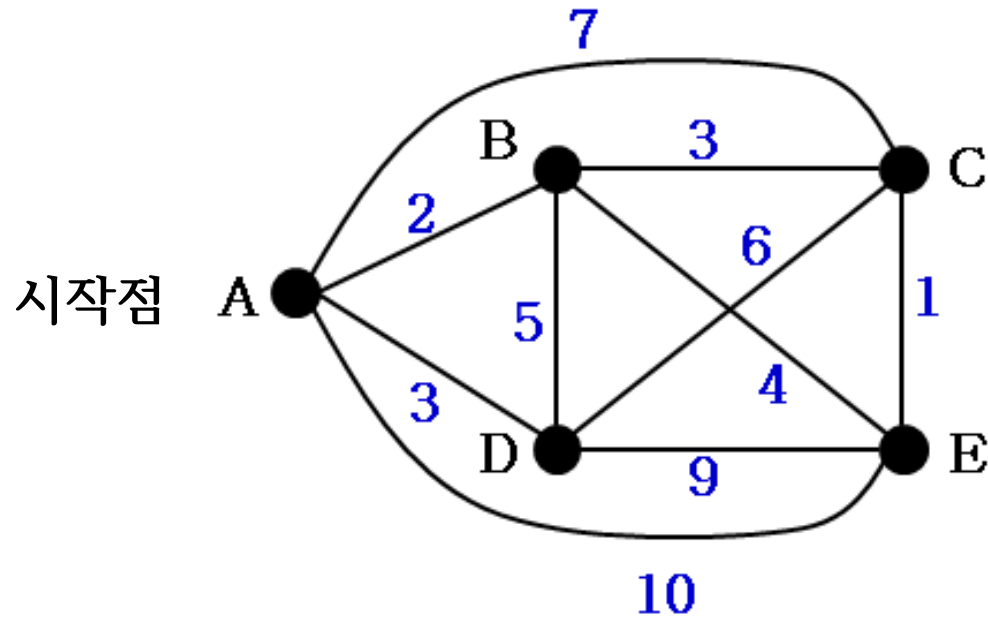
Branch-and-Bound Algorithm

- 점에 인접한 간선 중에서 2개의 가장 작은 가중치
 - 가중치 3인 간선으로 들어와서 가중치 2인 간선으로 나가든지 (왼쪽 그림)
 - 반대로 가중치 2인 간선으로 들어와서 가중치 3인 간선으로 나가든지 (오른쪽 그림)
- 두 경우 모두 최소의 비용으로 점을 방문함



Branch-and-Bound Algorithm

- 알고리즘 수행 과정
 - 5개의 점 (A, B, C, D, E)으로 된 그래프
 - 초기 상태 = [A]
 - Branch-and-Bound([A]) 호출



Branch-and-Bound Algorithm

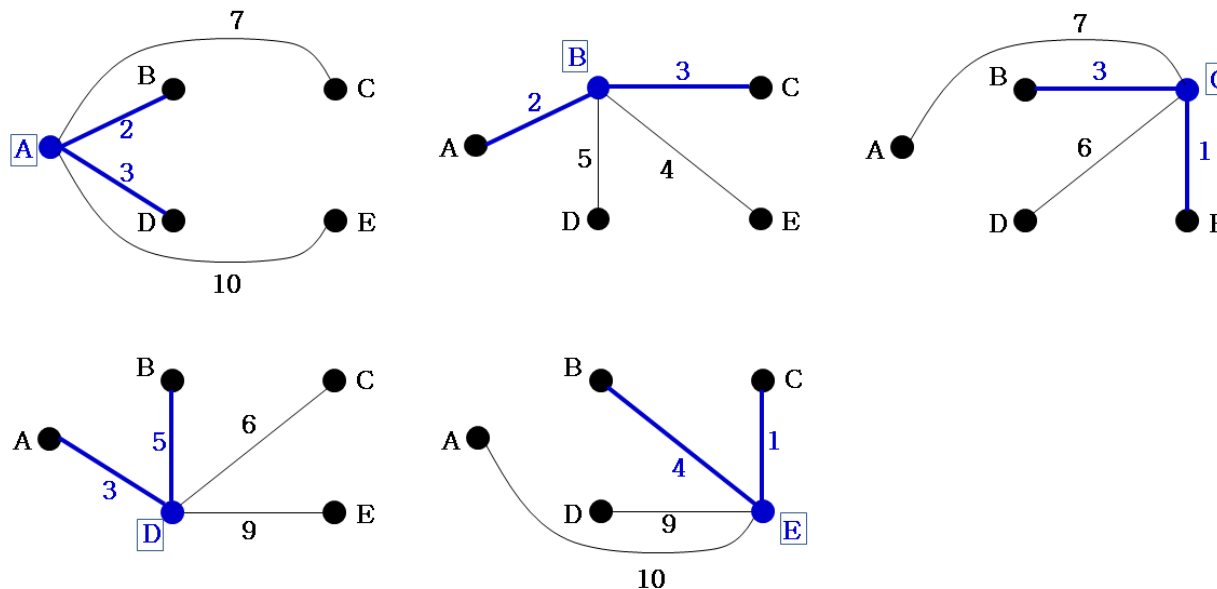
- 알고리즘 수행 과정

- 초기 상태 [A]의 한정값

- 초기 상태는 경로를 시작하기 전이므로, 각 점에 인접한 간선의 가중치 중에서 가장 작은 2개의 가중치의 합을 구한 다음에, 모든 점의 합의 $\frac{1}{2}$ 을 한정값으로 함

$$\begin{array}{ccccc} A & B & C & D & E \\ [(2+3) + (2+3) + (1+3) + (3+5) + (1+4)] \times \frac{1}{2} = 27/2 = 14 \end{array}$$

[A]
14



Branch-and-Bound Algorithm

- 알고리즘 수행 과정

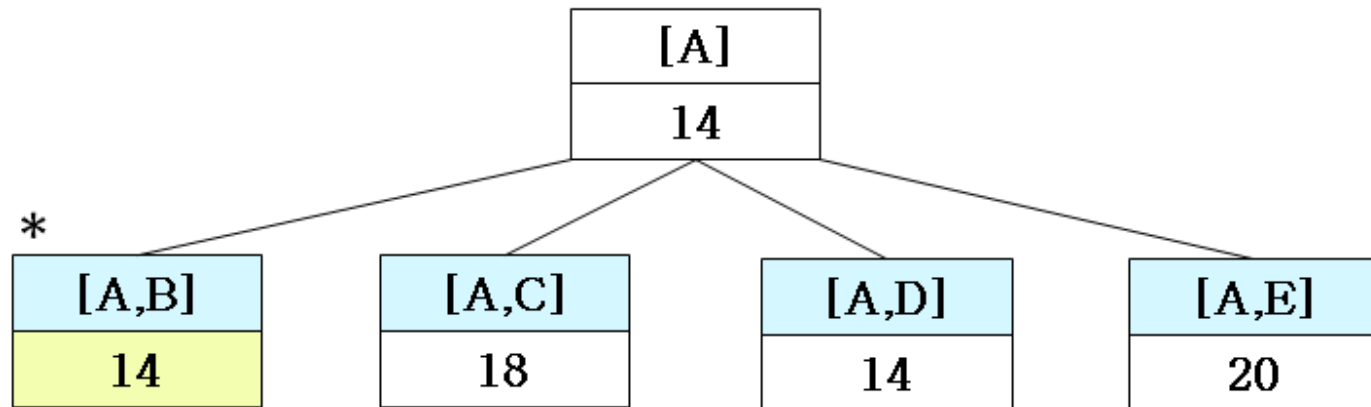
- $S_{\min} = [A]$

- activeNodes={S}, bestValue= ∞

- activeNodes 집합에 초기 상태 [A]만 있으므로, $S_{\min} = [A]$

- $S_{\min}$ (즉, 상태 [A])의 자식 노드 생성 및 한정값 계산

- 자식 노드는 점이 B인 상태 [A,B], C인 상태 [A,C], D인 상태 [A,D], E인 상태 [A,E]



Branch-and-Bound Algorithm

- 알고리즘 수행 과정

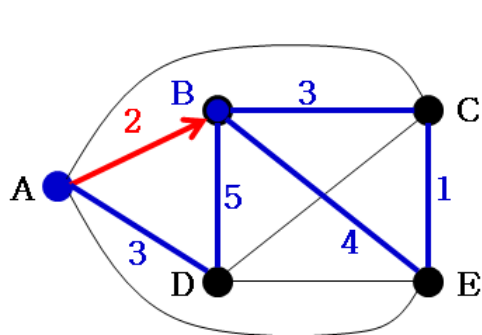
- 상태 $[A, B]$, $[A, C]$, $[A, D]$, $[A, E]$ 의 한정값

- $[A, B]$ 의 한정값 = $([2+3]+[2+3]+[1+3]+[3+5]+[1+4])/2 = 27/2 = 14$

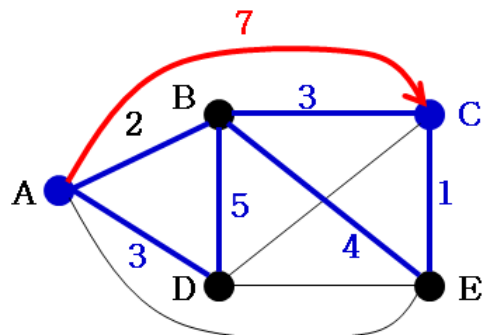
- $[A, C]$ 의 한정값 = $([2+7]+[2+3]+[1+7]+[3+5]+[1+4])/2 = 36/2 = 18$

- $[A, D]$ 의 한정값 = $([2+3]+[2+3]+[1+3]+[3+5]+[1+4])/2 = 27/2 = 14$

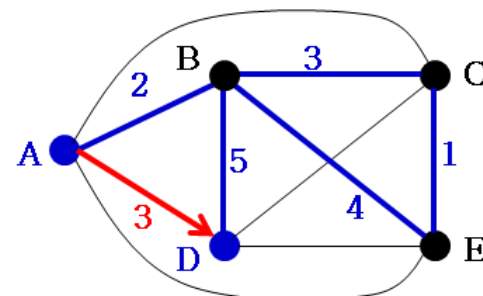
- $[A, E]$ 의 한정값 = $([2+10]+[2+3]+[1+3]+[3+5]+[1+10])/2 = 40/2 = 20$



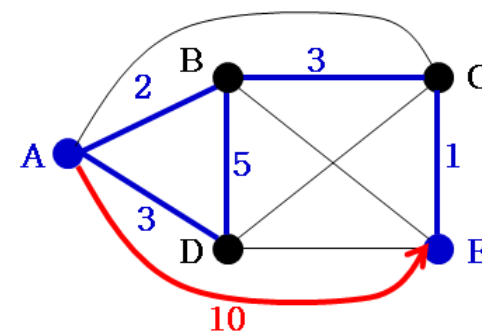
상태 $[A, B]$



상태 $[A, C]$



상태 $[A, D]$



상태 $[A, E]$

Branch-and-Bound Algorithm

- 알고리즘 수행 과정

- [A, B], [A, C], [A, D], [A, E] activeNodes에 추가

- activeNodes = {[A,B], [A,C], [A,D], [A,E]}

14 18 14 20

- 상태 [A, B]와 [A, D]가 동일한 최소의 한정값을 가지므로 임의의 $S_{\min} = [A, B]$

- activeNodes에서 [A, B]를 제거

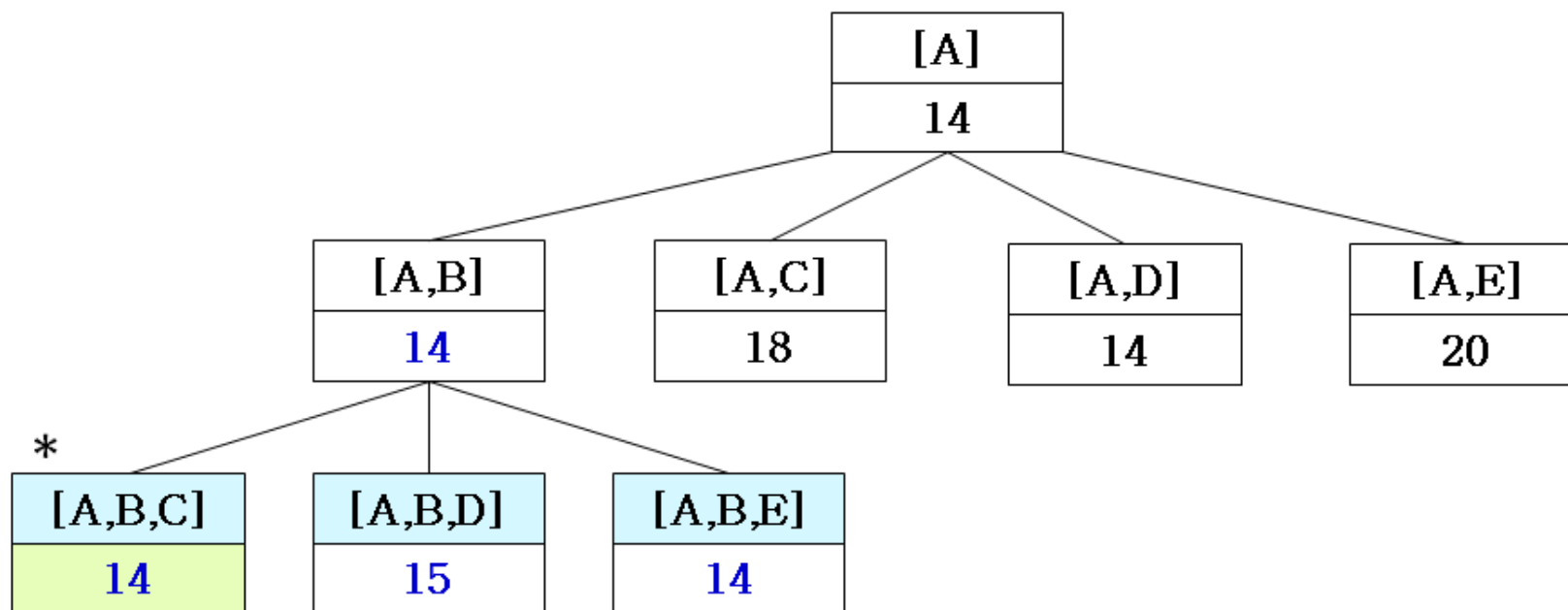
- activeNodes = {[A, C], [A, D], [A, E]}

Branch-and-Bound Algorithm

- 알고리즘 수행 과정

- [A, B]의 자식 상태 생성

- 자식 노드는 세 번째 방문하는 점이 C인 상태 [A, B, C], D인 상태 [A, B, D], E인 상태 [A, B, E]



Branch-and-Bound Algorithm

- 알고리즘 수행 과정

- [A, B, C], [A, B, D], [A, B, E] 한정값 계산

- [A, B, C]의 한정값

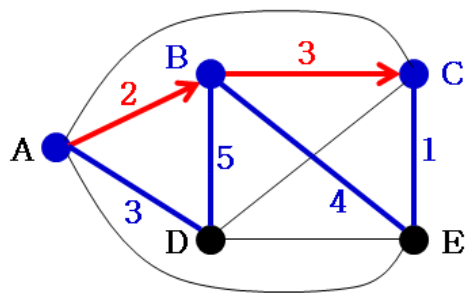
- $$([2+3]+[2+3]+[1+3]+[3+5]+[1+4])/2 = 27/2 = 14$$

- [A, B, D]의 한정값

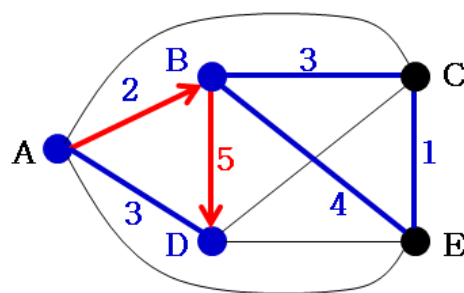
- $$([2+3]+[2+5]+[1+3]+[3+5]+[1+4])/2 = 29/2 = 15$$

- [A, B, E]의 한정값

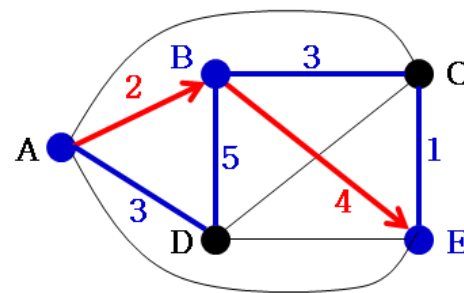
- $$([2+3]+[2+4]+[1+3]+[3+5]+[1+4])/2 = 28/2 = 14$$



상태 [A, B, C]



상태 [A, B, D]



상태 [A, B, E]

Branch-and-Bound Algorithm

- 알고리즘 수행 과정
 - [A, B, C], [A, B, D], [A, B, E] activeNodes에 추가
 - activeNodes = {[A, C], [A, D], [A, E], [A, B, C], [A, B, D], [A, B, E]}
- 18 14 20 14 15 14

Branch-and-Bound Algorithm

- 알고리즘 수행 과정

- $S_{\min} = [A, B, C]$

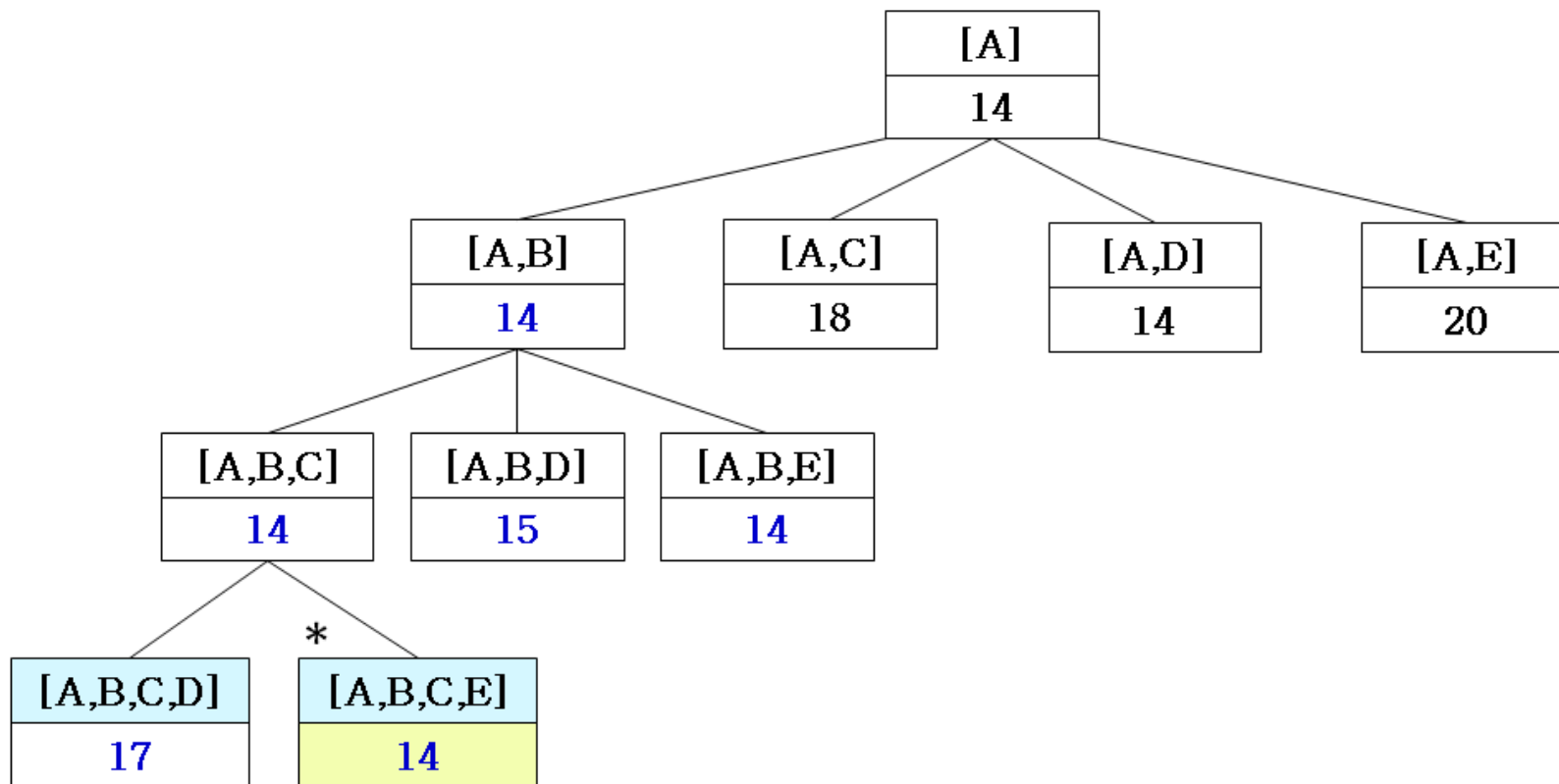
- 상태 $[A, B, C]$, $[A, B, E]$, $[A, D]$ 가 동일한 최소의 한정값을 가지므로 임의로 $S_{\min} = [A, B, C]$

- activeNodes에서 $[A, B, C]$ 제거

- $\text{activeNodes} = \{[A, C], [A, D], [A, E], [A, B, D], [A, B, E]\}$

Branch-and-Bound Algorithm

- 알고리즘 수행 과정
 - [A, B, C]의 자식 상태 생성
 - 자식 노드들은 네 번째 방문하는 점이 D인 상태 [A, B, C, D]와 E인 상태 [A, B, C, E]



Branch-and-Bound Algorithm

- 알고리즘 수행 과정
 - [A, B, C, D], [A, B, C, E]의 한정값
 - [A, B, C, D]의 한정값: $([2+3]+[2+3]+[6+3]+[3+6]+[1+4])/2 = 33/2 = 17$
 - [A, B, C, E]의 한정값: $([2+3]+[2+3]+[1+3]+[3+5]+[1+4])/2 = 27/2 = 14$
 - [A, B, C, D], [A, B, C, E] activeNodes에 추가
 - activeNodes = {[A, C], [A, D], [A, E], [A, B, D], [A, B, E], [A, B, C, D], [A, B, C, E]}
 - 18 14 20 15 14 17 14

Branch-and-Bound Algorithm

- 알고리즘 수행 과정

- $S_{\min} = [A, B, C, E]$

- 상태 $[A, B, C, E]$, $[A, B, E]$, $[A, D]$ 가 동일한 최소 한정값을 가지므로 임의로 $S_{\min} = [A, B, C, E]$

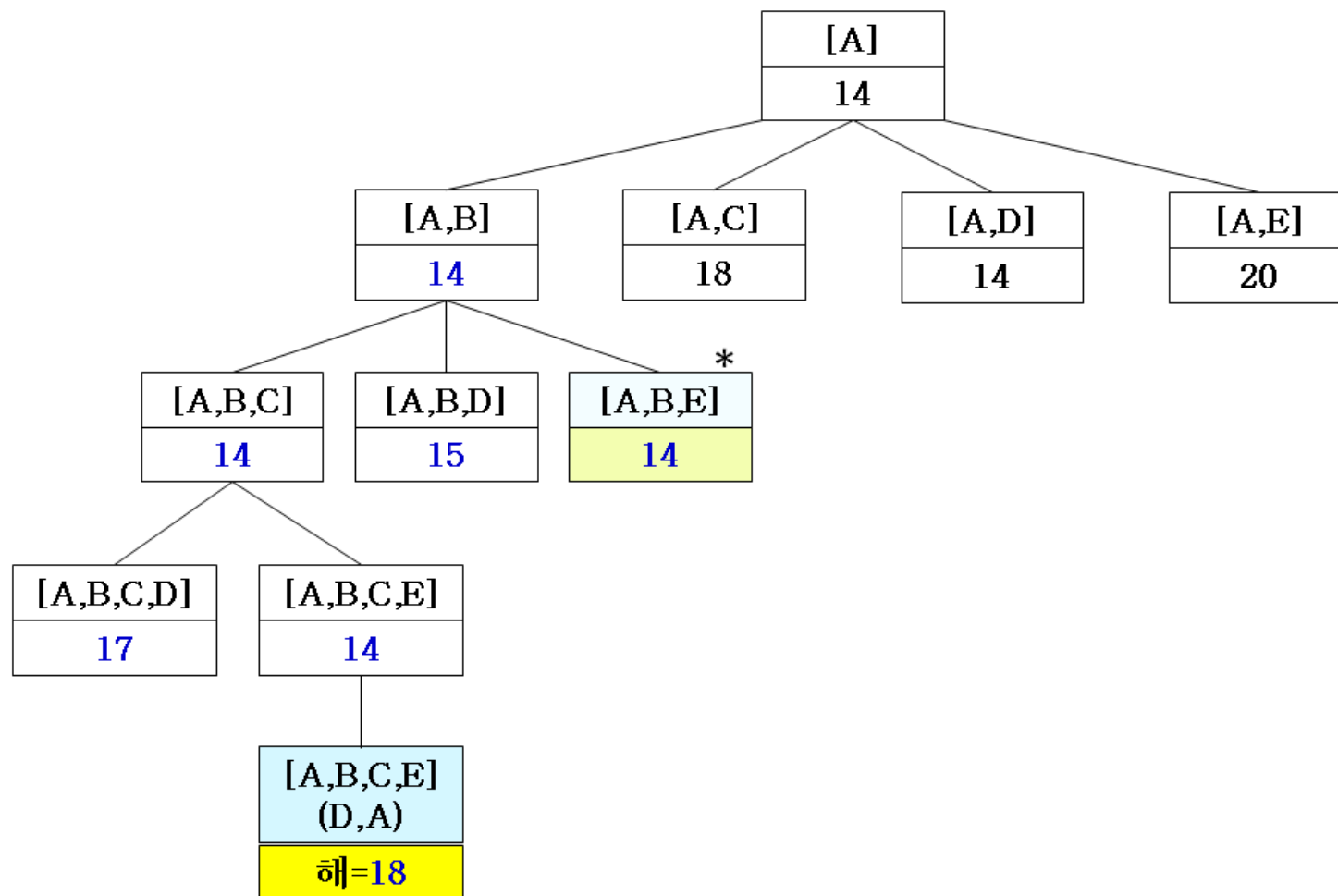
- activeNodes에서 $[A, B, C, E]$ 제거

- $\text{activeNodes} = \{[A, C], [A, D], [A, E], [A, B, D], [A, B, E], [A, B, C, D]\}$

Branch-and-Bound Algorithm

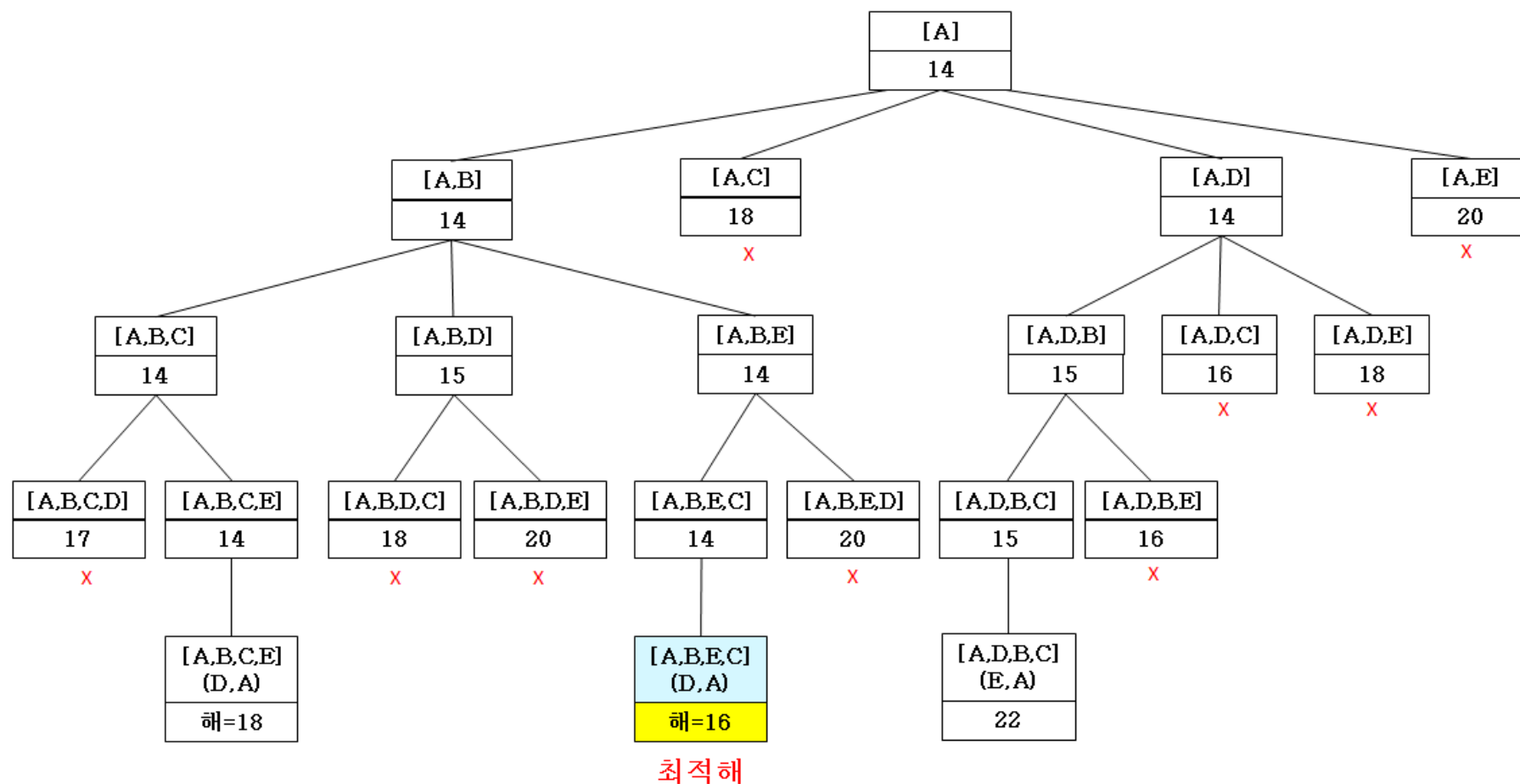
- 알고리즘 수행 과정

- bestValue=18, bestSolution=[A, B, C, E, D, A]



Branch-and-Bound Algorithm

- 알고리즘 수행 과정
 - 상태 [A, B, E]로부터 탐색 결과



Branch-and-Bound Algorithm

- Branch-and-bound vs. Backtracking
 - TSP를 위한 backtracking 알고리즘이 방문한 상태 공간 tree의 노드 수는 54개이나 branch-and-bound 알고리즘은 22개
 - 최적화 문제의 해를 탐색하는 데는 branch-and-bound 기법이 backtracking 기법보다 훨씬 우수한 성능을 보임
 - Branch-and-bound 알고리즘은 한정값을 사용하여 최적해가 없다고 판단되는 부분은 탐색을 하지 않고 최선 우선 탐색

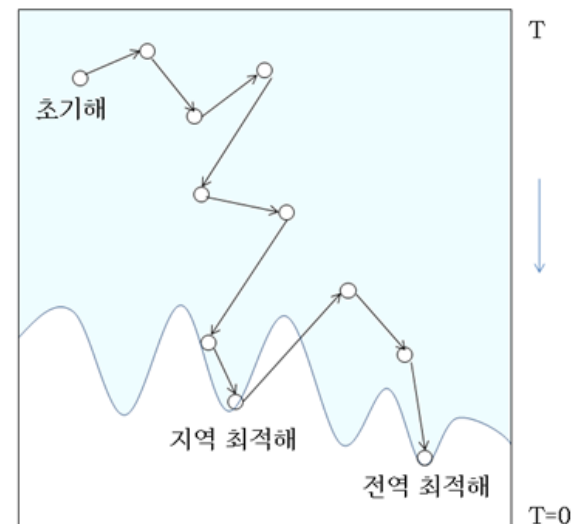
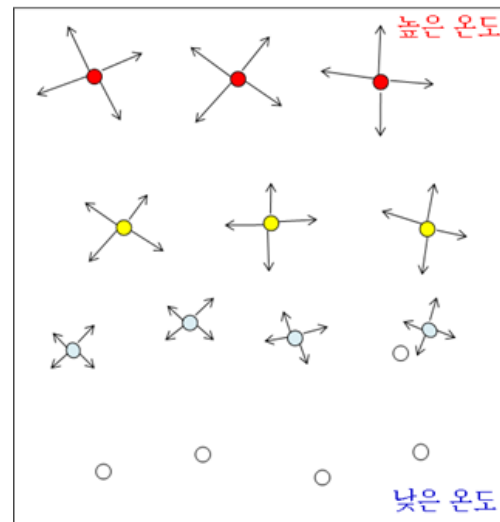
Simulated Annealing Algorithm

- Simulated Annealing Algorithm
 - 모의 담금질 (Simulated Annealing) 기법은 높은 온도에서 액체 상태인 물질이 온도가 점차 낮아지면서 결정체로 변하는 과정을 모방한 해 탐색 알고리즘
 - 용융 상태에서는 물질의 분자가 자유로이 움직이는데 이를 모방하여, 해를 탐색하는 과정도 특정한 패턴 없이 이루어짐
 - 온도가 점점 낮아지면 분자의 움직임이 점점 줄어들어 결정체가 되는데, 해 탐색 과정도 이와 유사하게 점점 더 규칙적인 방식으로 이루어짐

Simulated Annealing Algorithm

- 이웃해

- 이러한 방식으로 해를 탐색하려면, 후보해에 대해 이웃하는 해 (이웃해)를 정의하여야 함
- 아래의 오른쪽 그림에서 각 점은 후보해이고 아래쪽에 위치한 해가 위쪽에 있는 해보다 우수한 해임. 또한 2개의 후보해 사이의 화살표는 이 후보해들이 서로 이웃하는 관계임을 나타냄



Simulated Annealing Algorithm

- 탐색 과정
 - 높은 T에서의 초기 탐색은 최솟값을 찾는 데도 불구하고 확률 개념을 도입하여 현재 해의 이웃해 중에서 현재 해보다 '나쁜' 해로 (위 방향으로) 이동하는 자유로움을 보일 수 있음
 - T가 낮아지면서 점차 탐색은 아래 방향으로 향함
 - T가 낮아질수록 위 방향으로 이동하는 확률이 점차 작아짐
 - 그림에서 처음 도착한 골짜기 (지역 최적해, local optimum)에서 더 이상 아래로 탐색할 수 없는 상태에 이르렀을 때 '운 좋게' 위 방향으로 탐색하다가 전역 최적해 (global optimum)를 찾은 것을 보여줌

Simulated Annealing Algorithm

- Simulated Annealing Algorithm의 특성
 - Genetic algorithm과 마찬가지로 simulated annealing algorithm도 항상 전역 최적해를 찾아준다는 보장은 없음
 - 모의 담금질 기법의 또 하나의 특징은 하나의 초기해로부터 탐색이 진행된다는 것임. 반면에 genetic algorithm은 여러 개의 후보해를 한 세대로 하여 탐색을 수행

Simulated Annealing Algorithm

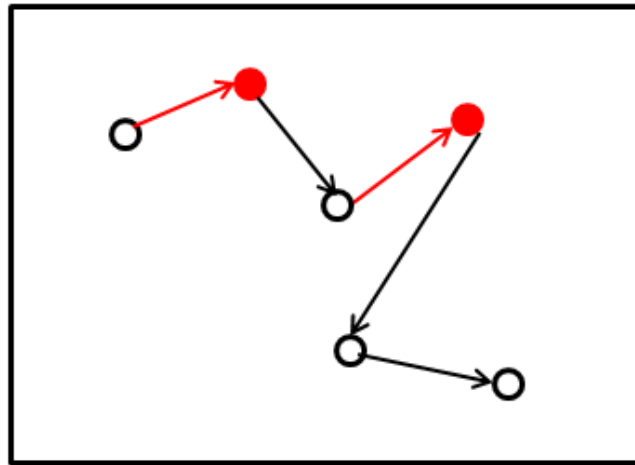
- 알고리즘

SimulatedAnnealing

1. 임의의 후보해 s 를 선택
2. 초기 T 를 정함
3. repeat
4. for $i = 1$ to K_T // K_T 는 T 에서의 for-loop 반복 횟수
5. s 의 이웃해 중에서 랜덤하게 하나의 해 s' 를 선택
6. $d = (s' \text{의 값}) - (s \text{의 값})$
7. if $d < 0$ // 이웃해인 s' 가 더 우수한 경우
8. $s \leftarrow s'$
9. else // s' 가 s 보다 우수하지 않은 경우
10. $q \leftarrow (0,1)$ 사이에서 랜덤하게 선택한 수
11. if $(q < p)$ // p 는 자유롭게 탐색할 확률
12. $s \leftarrow s'$
13. $T \leftarrow \alpha T$ // 1보다 작은 상수 α 를 T 에 곱하여 새로운 T 를 계산
14. until 종료 조건이 만족될 때까지
15. return s

Simulated Annealing Algorithm

- 자유롭게 탐색할 확률 p
 - Line 9~11: s' 가 s 보다 우수하지 않더라도 0~1 사이에서 랜덤하게 선택한 수 q 가 확률 p 보다 작으면, s' 가 현재 해인 s 가 될 기회를 줌
 - 이 기회가 그림에서 최소값을 찾는데도 불구하고 위쪽에 위치한 이웃해로 탐색을 진행함
 - p 는 자유롭게 탐색할 확률



Simulated Annealing Algorithm

- 냉각율

- Line 12: T를 일정 비율 α 로 감소시킴. 실제로 $0.8 \leq \alpha \leq 0.99$ 범위에서 미리 정한 냉각율 α (cooling ratio)를 T에 곱하여 새로운 T를 계산
 - 일반적으로 0.99에 가까운 수로 선택

Simulated Annealing Algorithm

- 확률 p 조절
 - Simulated Annealing Algorithm은 T 가 높은 때부터 점점 낮아지는 것을 확률 p 에 반영시켜서 초기에는 탐색이 자유롭다가 점점 규칙적이 되도록 함
 - 확률 p 는 T 에 따라서 변해야 함
 - T 가 높을 때는, p 를 크게 하고,
 - T 가 0이 되면, p 를 0으로 만들어서 나쁜 이웃해 s' 가 s 가 되지 못하도록 함
 - s' 와 s 의 값의 차이 d 에 따라 p 조절
 - d 값이 크면 p 를 작게 하고
 - d 값이 작으면 p 를 크게 함
 - 이렇게 하는 이유는 차이가 크에도 불구하고 p 를 크게 하면 그 동안 탐색한 결과가 무시되어 랜덤하게 탐색하는 결과를 낳기 때문

Simulated Annealing Algorithm

- 확률 p
 - 두 가지 요소를 종합한 확률 p
 - $p = 1 / e^{d/T} = e^{-d/T}$
- T 는 큰 값에서 0까지 변하고, d 는 s' 과 s 의 값의 차이

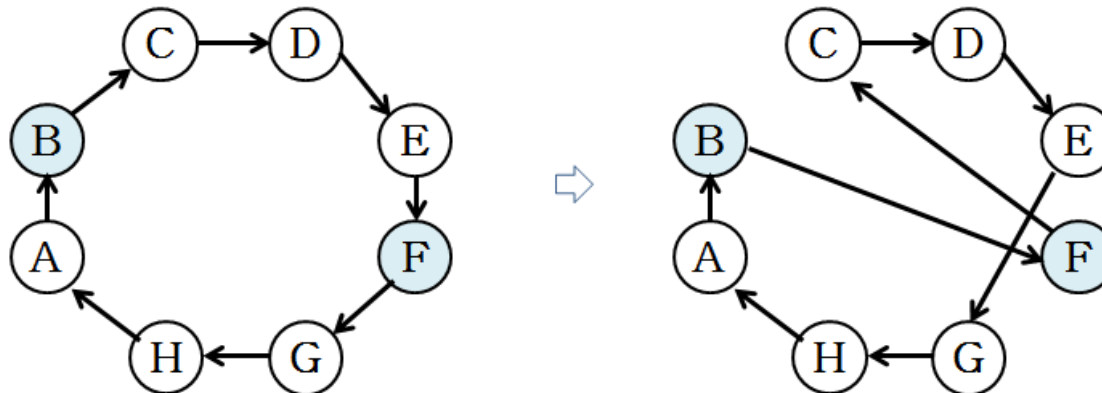
Simulated Annealing Algorithm

- 이웃해 정의
 - TSP의 이웃해 정의 3가지 예
 - 삽입 (Insertion)
 - 교환 (Switching)
 - 반전 (Inversion)

Simulated Annealing Algorithm

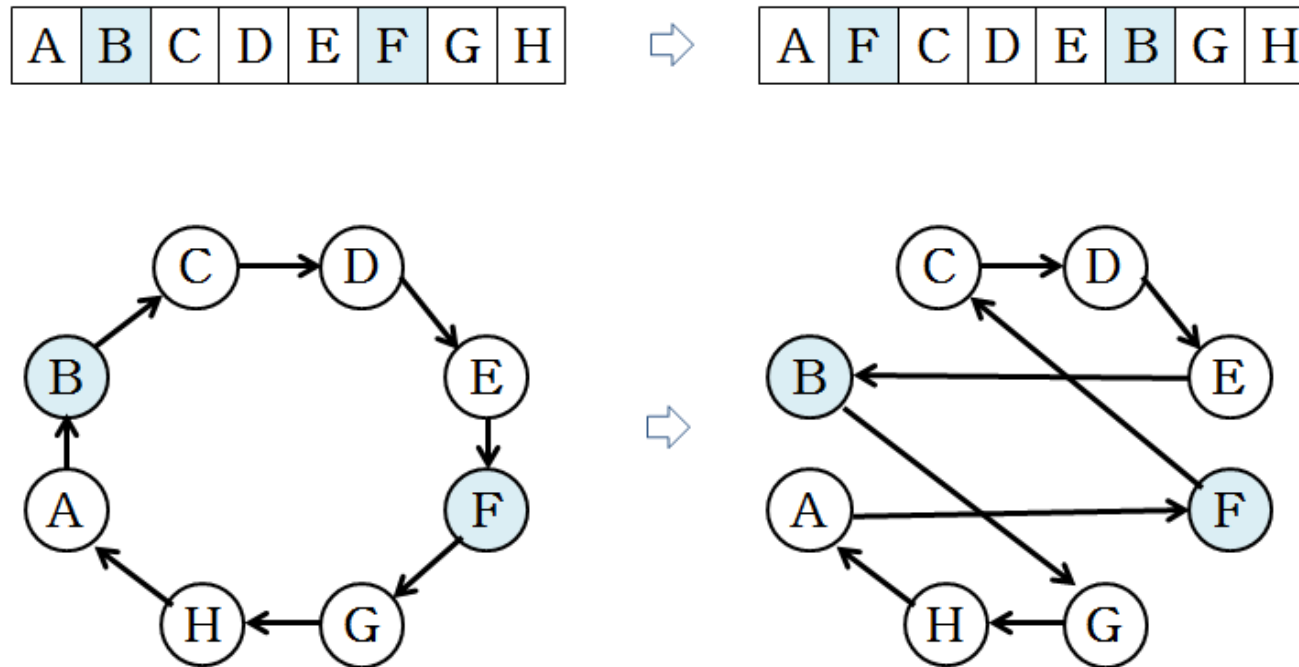
- 삽입 (Insertion)

- 2개의 도시를 랜덤하게 선택한 후에, 두 번째 도시를 첫 번째 도시 옆으로 옮기고, 두 도시 사이의 도시들은 오른쪽으로 1칸씩 이동
- 도시 B와 F가 랜덤하게 선택되었다면, F가 B의 바로 오른쪽으로 이동한 후, B와 F 사이의 C, D, E를 각각 오른쪽으로 1칸씩 이동



Simulated Annealing Algorithm

- 교환 (Switching)
 - 2개의 도시를 랜덤하게 선택한 후에, 그 도시들의 위치를 서로 바꿈
 - 도시 B와 F가 랜덤하게 선택되었다면, B와 F의 자리를 서로 바꿈



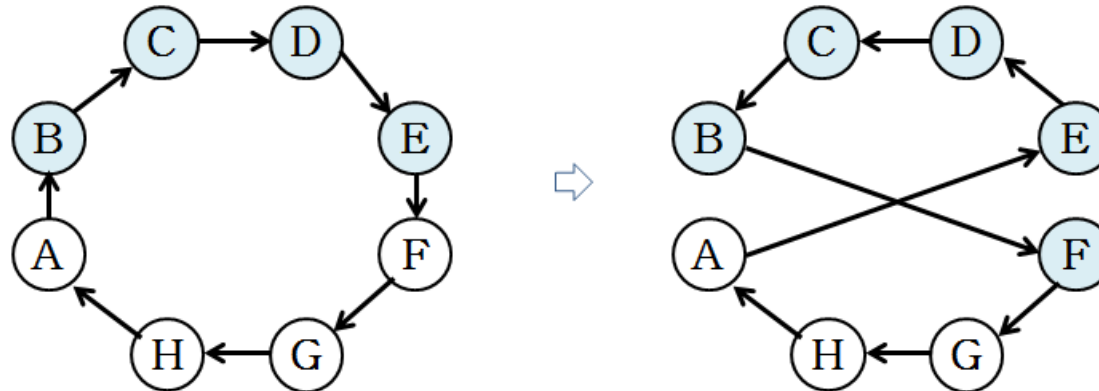
Simulated Annealing Algorithm

- 반전 (Inversion)

- 2개의 도시를 랜덤하게 선택한 후에, 그 두 도시 사이의 도시를 역순으로 만듦.

- 단, 선택된 두 도시도 반전에 포함시킴

- 도시 B와 E가 랜덤하게 선택되었다면, [B C D E]가 역순으로 [E D C B]로 바뀜



요약

- Backtracking 기법은 해를 찾는 도중에 '막히면' 되돌아가서 다시 해를 찾아 가는 기법으로 상태 공간 tree에서 **깊이 우선 탐색 (Depth First Search)**으로 해를 찾는 알고리즘
- Backtracking 기법의 time complexity는 상태 공간 tree의 노드 수에 비례하고, 이는 모든 경우를 다 검사하여 해를 찾는 완전 탐색의 time complexity와 같음. 그러나 일반적으로 backtracking 기법은 '가지치기'를 하므로 완전 탐색보다 훨씬 효율적임
- Branch-and-bound 기법은 상태 공간 tree의 각 노드 (상태)에 특정한 값 (한정값)을 부여하고, 노드의 한정값을 활용하여 가지치기를 함으로서 backtracking 기법보다 빠르게 해를 찾음

요약

- Branch-and-bound 기법에서는 가장 우수한 한정값을 가진 노드를 먼저 탐색하는 **최선 우선 탐색 (Best First Search)**으로 해를 찾음
- Genetic algorithm은 다윈의 진화론으로부터 고안된 해 탐색 알고리즘임. '적자생존' 개념을 최적화 문제를 해결하는 데에 적용한 것임
- Genetic algorithm은 여러 개의 해를 임의로 생성하여 이들에 대해 선택, 교차, 돌연변이 연산을 반복 수행하여 마지막에 가장 우수한 해를 리턴

요약

- Genetic algorithm은 문제의 최적해를 알 수 없고, 기존의 어느 알고리즘으로도 해결하기 어려운 경우에, 최적해에 가까운 해를 찾는데 매우 적절한 알고리즘
- 모의 담금질 (Simulated Annealing) 알고리즘은 높은 온도에서 액체 상태인 물질이 온도가 점차 낮아지면서 결정체로 변하는 과정을 모방한 해 탐색 알고리즘
- Genetic algorithm과 마찬가지로 Simulated annealing algorithm도 항상 전역 최적해를 찾아준다는 보장은 없음